

PROMPT 설계 & CONTEXT ENGINEERING

B2B AI 서비스 기획 & 프롬프트 설계 실습

3인 팀 실습 사전 학습 자료 - 서비스 기획부터 프롬프트 스크립트 작성까지

대상 기법 : CoT · Self-Consistency · ReAct · Harness Engineering · Memory & Compaction

2026 임성열 교수 Ph.D.

Prompt • Context • Harness



이 실습에서 배우는 세 가지

1

서비스 기획력

이해관계자와 그들이 얻는 가치를 스스로 정의하고, B2B 맥락에서 타당한 서비스를 설계하는 연습입니다.

2

기법 선택력

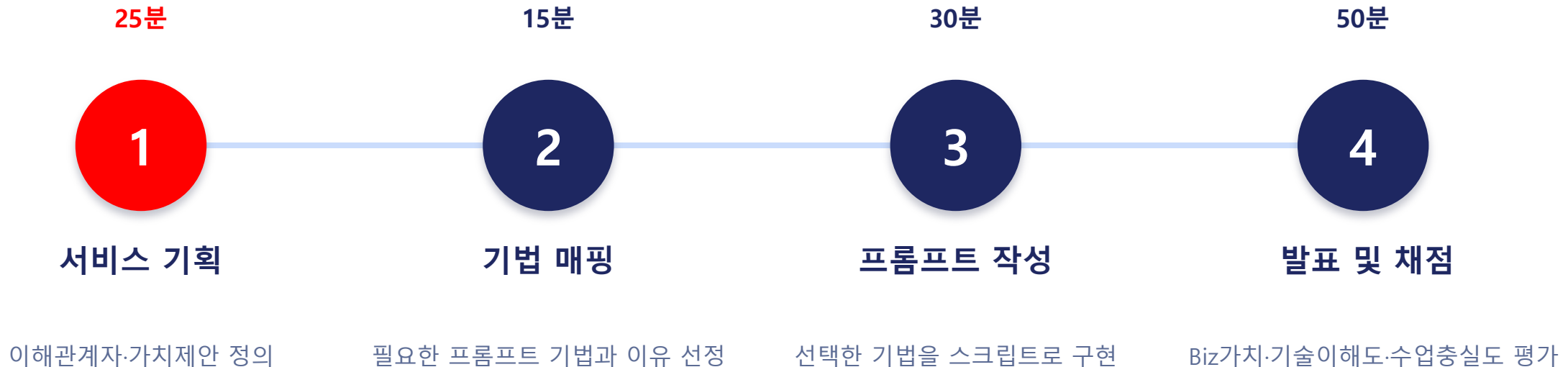
서비스 특성에 따라 CoT·SC·ReAct·Harness·Memory 중 어떤 기법이 왜 필요한지 근거를 세워 고르는 연습입니다.

3

실전 프롬프트 작성력

선택한 기법을 말로만 설명하지 않고, 실제로 동작하는 프롬프트 스크립트로 구현하는 연습입니다.

실습 진행 순서 (총 120분 예상)



팀 구성 : 3인 1팀 · 산출물 : 서비스 기획서 + 프롬프트 스크립트, 발표 자료에 팀원 이름 및 역할 기재

실습에 활용할 핵심 프롬프트 기법

CoT

Chain of Thought

단계별 사고 과정을 명시해 추론 정확도를 높이는 기법

SC

Self-Consistency

같은 문제를 여러 경로로 풀어 답을 교차검증하는 기법

ReAct

Reason + Act

추론과 도구 호출(검색·조회)을 번갈아 수행하는 기법

HE

Harness Engineering

거대한 단일 프롬프트 대신 단계별 파이프라인으로 작업을 구조화하는 기법

MC

Memory & Compaction

길어지는 대화·컨텍스트에서 무엇을 기억하고 무엇을 요약할지 설계하는 기법

오늘 실습은 이 5가지 기법을
3개 서비스 템플릿에 직접 적용

Chain of Thought (CoT)

단계별 사고를 명시해 추론 정확도를 높이는 기법

핵심 아이디어

정답만 요청하지 않고, 중간 추론 과정을 순서대로 드러내도록 지시하는 기법입니다. 모델이 스스로 단계를 밟으며 점검할 기회가 생겨 다단계 판단의 정확도가 올라갑니다.

왜 효과적인가?

복잡한 조건 분기(예 : 구간별 요율 적용)에서 한 번에 답을 내면 중간 단계를 건너뛰어 오답이 나기 쉽습니다. CoT는 이 건너뛸을 프롬프트 차원에서 막습니다.

동작 흐름



Chain of Thought (CoT)

언제 쓰고, 언제 조심해야 하는가?

이럴 때 적합

- 조건 분기가 있는 계산 (SLA 크레딧 구간 판정)
- 여러 사실을 종합해야 하는 판단 (계약 조항 해석)
- 실수하면 비용이 큰 의사결정을 보조할 때

이럴 땐 주의

- 단순 사실 조회에 강제하면 불필요하게 장황해짐
- 모든 질문에 기계적으로 '단계별로 생각해'만 붙이면 프롬프트만 비대해짐
- 추론 단계 수는 문제 복잡도에 맞게 설계해야 함

Chain of Thought (CoT)

실제 프롬프트 패턴 예시 · 적용 대상 : 템플릿 1.3

✕ 일반 지시 : "이번 달 크레딧 얼마야?"

✓ CoT 적용

템플릿 1.3

다음 순서로 생각한 뒤 답하세요.

- 1) 이번 달 가동률을 확인한다
- 2) FAQ의 SLA 크레딧 구간표에서 해당 구간을 찾는다
- 3) 구독료에 구간별 %를 곱해 크레딧 금액을 계산한다
- 4) 이전 장애 건과 별개인지 확인한다
- 5) 위 근거를 바탕으로 최종 답변을 작성한다

Chain of Thought (CoT)

실습 시 이 기법을 확인하는 방법

- 프롬프트에 '단계별로' 같은 명시적 추론 지시가 있는가?
- 최종 답 이전에 중간 추론이 실제로 드러나는가?
- 추론 단계 수가 문제 복잡도에 비해 과하거나 부족하지 않은가?

이 기법은 템플릿 1.3 실습에서 직접 확인할 수 있습니다.

Self-Consistency (SC)

같은 문제를 여러 경로로 풀어 답을 교차검증하는 기법

핵심 아이디어

동일한 문제를 서로 다른 관점·경로로 여러 번 풀게 한 뒤, 공통적으로 나온 답만 채택하거나 다수결로 최종 답을 정하는 기법입니다.

왜 효과적인가

한 번의 추론 경로만 믿으면 우연한 오류를 그대로 최종 답으로 채택할 위험이 있습니다. 여러 경로를 비교하면 오류가 자연스럽게 걸러집니다.

동작 흐름



Self-Consistency (SC)

언제 쓰고, 언제 조심해야 하는가?

이럴 때 적합

- '맞다/틀리다'가 명확히 갈리는 고위험 판단 (계약 리스크 여부)
- 오탐 비용이 큰 이상탐지 (이상거래 최종 확정)
- 법무·재무·운영처럼 여러 관점이 섞인 평가

이럴 땐 주의

- 호출 횟수(N배)만큼 비용·시간이 증가함
- 다양성이 필요한 창작성 답변에는 부적합
- 몇 번, 어떤 기준으로 교차검증할지 프롬프트에 명시해야 함

Self-Consistency (SC)

실제 프롬프트 패턴 예시 · 적용 대상 : 템플릿 2

✕ **가짜 SC** : "확실히 다시 한번 확인해봐" (경로 분리 없음)

✓ SC 적용

템플릿 2

이 계약서의 각 조항을 아래 3가지 관점에서
각각 독립적으로 평가하세요.

1) 법무 리스크 2) 재무 리스크 3) 운영 리스크
세 관점에서 공통적으로 지적된 조항만
최종 리스크 목록에 포함하세요.

Self-Consistency (SC)

실습 시 이 기법을 확인하는 방법

- 실제로 여러 경로·관점으로 나눠 풀게 했는가?
- 경로 간 불일치 시 처리 규칙(다수결 등)이 있는가?
- 비용 대비 정확도 향상을 설명할 수 있는가?

이 기법은 템플릿 2 실습에서 직접 확인할 수 있습니다.

ReAct (Reason + Act)

추론과 도구 호출(검색·조회)을 번갈아 수행하는 기법

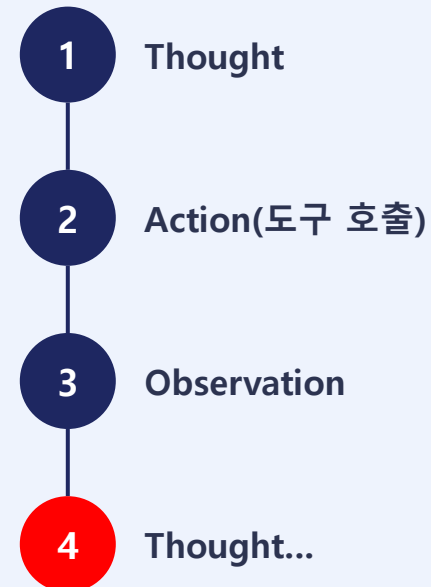
핵심 아이디어

모델이 '생각한다 → 도구를 호출해 조회한다 → 결과를 관찰한다 → 다시 생각한다'를 반복하며 답에 도달하는 기법입니다.

왜 효과적인가

모델 스스로 아는 지식만으로는 최신 정책이나 사내 데이터를 알 수 없습니다. ReAct는 외부 조회 결과를 추론에 실시간으로 반영합니다.

동작 흐름



ReAct (Reason + Act)

언제 쓰고, 언제 조심해야 하는가?

이럴 때 적합

- 사내 FAQ·정책 문서 조회가 필요한 경우
- 대량 거래 데이터에서 조건에 맞는 행을 필터링해야 하는 경우
- 실시간 정보가 답의 정확도를 좌우하는 경우

이럴 땐 주의

- 이미 아는 지식만으로 충분한 질문엔 불필요한 지연만 발생
- 조회 결과(Observation)를 검증 없이 그대로 믿으면 안 됨
- 무한 반복을 막는 최대 반복 횟수 설정이 필요함

ReAct (Reason + Act)

실제 프롬프트 패턴 예시 · 적용 대상: 템플릿 1.3

✕ 도구 없이 추측 : "동일산업 거래가 이상한 것 같아" (근거 없는 단정)

✓ REACT 적용

템플릿 1.3

Thought: 먼저 거래처별 평균 금액을 확인해야 한다.

Action: 거래처 V009의 최근 10건 평균 금액을 조회하라.

Observation: 평균 168,000원

Thought: 이번 건(6,720,000원)은 평균 대비 약 40배로
이례적이다. 이상거래로 판단한다.

ReAct (Reason + Act)

실습 시 이 기법을 확인하는 방법

- 실제 조회·도구 호출 단계가 프롬프트에 설계되어 있는가?
- 조회 결과를 다음 추론에 반영하는 루프가 있는가?
- 최대 반복 횟수 등 무한루프 방지 장치가 있는가?

이 기법은 템플릿 1.3 실습에서 직접 확인할 수 있습니다.

Harness Engineering

단일 거대 프롬프트 대신 파이프라인으로 작업을 구조화하는 기법

핵심 아이디어

하나의 프롬프트에 모든 지시를 몰아넣는 대신, 추출·분석·검증처럼 역할이 다른 단계로 나누어 각 단계의 입력과 출력을 명확히 정의하는 기법입니다.

왜 효과적인가?

단일 프롬프트는 지시가 길어질수록 일부 지시를 놓치기 쉽습니다. 단계를 나누면 각 단계가 한 가지 일에만 집중해 품질이 안정됩니다.

동작 흐름



Harness Engineering

언제 쓰고, 언제 조심해야 하는가?

이럴 때 적합

- 여러 하위 작업이 섞인 복잡한 문서 처리 (계약서 검토)
- 결과물의 신뢰도가 중요해 검증 단계가 꼭 필요한 경우
- 산출물을 재사용 가능한 형태로 남기고 싶은 경우

이럴 땐 주의

- 단순 1턴 질문에 과도한 파이프라인은 지연·복잡성만 증가
- 단계가 많을수록 정확도는 오르지만 비용·시간도 증가
- 왜 이렇게 단계를 나눴는지 근거를 설명할 수 있어야 함

Harness Engineering

실제 프롬프트 패턴 예시 · 적용 대상 : 템플릿 2

✕ 단일 프롬프트 : "이 계약서 검토해서 문제점 다 알려줘" (한 번에 전부 요청)

✓ HE 적용 (3단계)

템플릿 2

1단계: 계약서에서 조항별 텍스트를 표로 추출하라.

2단계: 추출된 각 조항을 리스크 유형별로 분류하라.

3단계: 분류된 리스크에 근거와 함께 요약 리포트를 작성하라.

(각 단계 출력이 다음 단계의 입력이 됨)

Harness Engineering

실습 시 이 기법을 확인하는 방법

- 프롬프트가 실제로 2개 이상 단계로 분리되어 있는가?
- 각 단계의 입력·출력이 명확히 정의되어 있는가?
- 단계를 나눈 이유를 설명할 수 있는가?

이 기법은 템플릿 2 실습에서 직접 확인할 수 있습니다.

Memory & Compaction

길어지는 대화에서 무엇을 기억하고 무엇을 요약할지 설계하는 기법

핵심 아이디어

대화·컨텍스트가 길어질 때, 반드시 보존해야 할 핵심 정보와 요약해도 되는 정보를 구분해 다음 턴의 컨텍스트를 구성하는 기법입니다.

왜 효과적인가

모든 것을 다 기억하려 하면 컨텍스트가 무한히 길어지고, 무작정 요약하면 핵심 정보를 잃습니다. 무엇을 보존할지 명시적으로 설계해야 합니다.

동작 흐름



Memory & Compaction

언제 쓰고, 언제 조심해야 하는가?

이럴 때 적합

- 여러 턴에 걸친 상담 이력 (헬프데스크 대화)
- 장기 세션으로 운영되는 분석 에이전트
- 이전 이력이 이후 판단의 전제가 되는 모든 서비스

이럴 땐 주의

- 1회성 단발 질문에는 불필요
- 무엇을 압축할지 잘못 판단하면 핵심 정보(담당자 변경, 이전 크레딧 이력)를 잃어버림
- 긴 대화일수록 정보 유실 여부를 직접 테스트해봐야 함

Memory & Compaction

실제 프롬프트 패턴 예시 · 적용 대상: 템플릿 1

✕ 막연한 지시 : "이전 내용 잘 기억해서 답해줘"

✓ MC 적용

템플릿 1

대화가 길어지면 다음 정보는 반드시 유지하세요.

- 고객 플랜/구독료 · 미처리 요청
- 최근 담당자명 · 최근 크레딧 이력

그 외 단순 인사 및 일반 안내는 한 줄로 요약해도 됩니다.

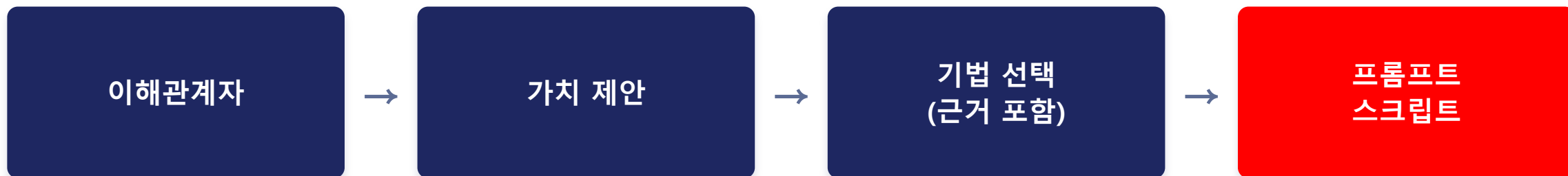
Memory & Compaction

실습 시 이 기법을 확인하는 방법

- 무엇을 기억할지 보존 목록(retention list)을 명시했는가?
- 긴 대화 테스트에서 초반 정보를 끝까지 정확히 유지하는가?
- 압축 과정에서 핵심 정보 손실이 없는가?

이 기법은 템플릿 1 실습에서 직접 확인할 수 있습니다.

서비스 기획 → 프롬프트 설계 프레임



이 4단계 산출물이 아래 3가지 평가 기준으로 그대로 연결

Biz 가치

이해관계자가 명확하고, 놓쳤을 때 손해를 설명할 수 있는가?

기술 이해도

기법을 왜 그 조합으로 썼는지 근거가 있는가?

수업 충실도

기법 이름과 실제 프롬프트 구조가 일치하는가?

1

고객 대응형 - 사내 헬프데스크 AI

대화·메모리 중심 서비스

이해관계자

실사용자 (임직원/고객사 담당자) · 구매 의사결정자 (고객사 임원) · 운영팀 (CS팀)

가치 제안

반복 문의 응대 시간 단축과 24시간 대응. 긴 대화에서도 핵심 정보를 놓치지 않는 것이 품질의 핵심

핵심 기법

Memory & Compaction (대화 이력 요약·보존) · ReAct (사내 FAQ 조회) · CoT (예외 판단 보조)

제공 실습자료

가상_사내정책FAQ_템플릿1.docx (조회 대상 지식베이스) · 가상_고객상담이력_템플릿1.docx (28턴 상담이력, 핵심 사실이 흩어져 있음)

2 내부 업무 자동화형 - 제안서·계약서 검토 어시스턴트

구조화·검증 중심 서비스

이해관계자

실무자 (영업/법무 담당자) · 승인권자 (팀장·임원) · 고객사 (문서 품질 영향)

가치 제안

문서 작성·검토 시간 절감과 리스크 조항 누락 방지. 하나의 프롬프트로 다 처리하지 않는 것이 관건

핵심 기법

Harness Engineering (추출→초안→검증 파이프라인 분리) · Self-Consistency (리스크 판단 교차검증)

제공 실습자료

가상_B2B_계약서_템플릿2.docx - 자동갱신·IP귀속·재위탁·손해배상·해지권 등 리스크 조항 5개가 의도적으로 삽입된 가상 계약서

3

의사결정 지원형 - 공급망·재무 이상탐지 에이전트

탐색·신뢰도 중심 서비스

이해관계자

분석 실무자 (데이터 담당자) · 최종 의사결정자 (경영진) · IT/보안팀 (접근 통제)

가치 제안

대량 데이터에서 이상 패턴을 신속히 발견하고 근거를 제시. 틀렸을 때 책임이 큰 서비스이므로 검증 장치가 필수

핵심 기법

ReAct (데이터 다단계 조화·필터링) · CoT (판단 근거 노출) · Self-Consistency (오탐 감소를 위한 재확인)

제공 실습자료

가상_거래데이터셋_템플릿3.xlsx - 발주 거래 151건 중 중복발주·이상고액거래·휴일새벽거래 등 이상거래 5유형이 삽입된 가상 데이터셋

평가 기준 : 발표 시 확인 포인트

Biz 가치

- 이해관계자가 구체적으로 정의되어 있는가?
- 이 서비스가 없으면 어떤 손해·비효율이 발생하는지 설명하는가?
- 가치 제안이 막연하지 않고 측정 가능한가?

기술 이해도

- 단일 프롬프트가 아닌 단계 분리 (Harness) 설계가 있는가?
- 기법을 선택한 이유를 서비스 특성과 연결해 설명하는가?
- Self-Consistency 등으로 재확인 장치를 두었는가?

수업 충실도

- 기법 이름과 실제 프롬프트 구조가 일치하는가?
- 예: CoT라 주장하며 단순 지시문만 있으면 감점
- 배운 기법을 최소 2개 이상 실제로 조합했는가?

BEFORE YOU START

실습 전 체크리스트

- 1 템플릿 3종 실습자료(FAQ+상담이력 / 계약서 / 거래데이터셋) 팀별 사전 배포 확인
- 2 팀별 LLM 접속 환경(계정·API) 및 노트북 점검
- 3 기획 프레임(이해관계자-가치-기법-프롬프트) 작성용 워크시트 배포
- 4 발표 시간 및 평가 기준(Biz가치·기술이해도·수업충실도) 공지



Good luck, and design something the stakeholders will actually want to use.